

Threads & Concorrenza

Obiettivi

- Introdurre la nozione di thread
- Multithreaded programming
- Supporti per threads in Windows e Linux

Motivazioni

- Ad un generico processo, sono associati i seguenti dati e le seguenti informazioni:
 - codice del programma in esecuzione
 - un'area di memoria contenente le strutture dati dichiarate nel programma in esecuzione
 - file aperti
 - stack(per le chiamate di procedure e funzioni)
 - contenuto dei registri della CPU

Motivazioni

- Consideriamo due processi che devono lavorare sugli stessi dati.
- Come possono fare, se ogni processo ha la propria area dati (ossia, gli spazi di indirizzamento sono separati)?
 - i dati possono essere scambiati mediante messaggi (pipe, FIFO) o tenuti in memoria condivisa
 - i dati possono essere tenuti in un file che viene acceduto a turno dai due processi.
- Non sarebbe comodo poter avere processi che possano automaticamente lavorare sugli stessi dati, senza usare meccanismi espliciti di condivisione/comunicazione?

Motivazioni

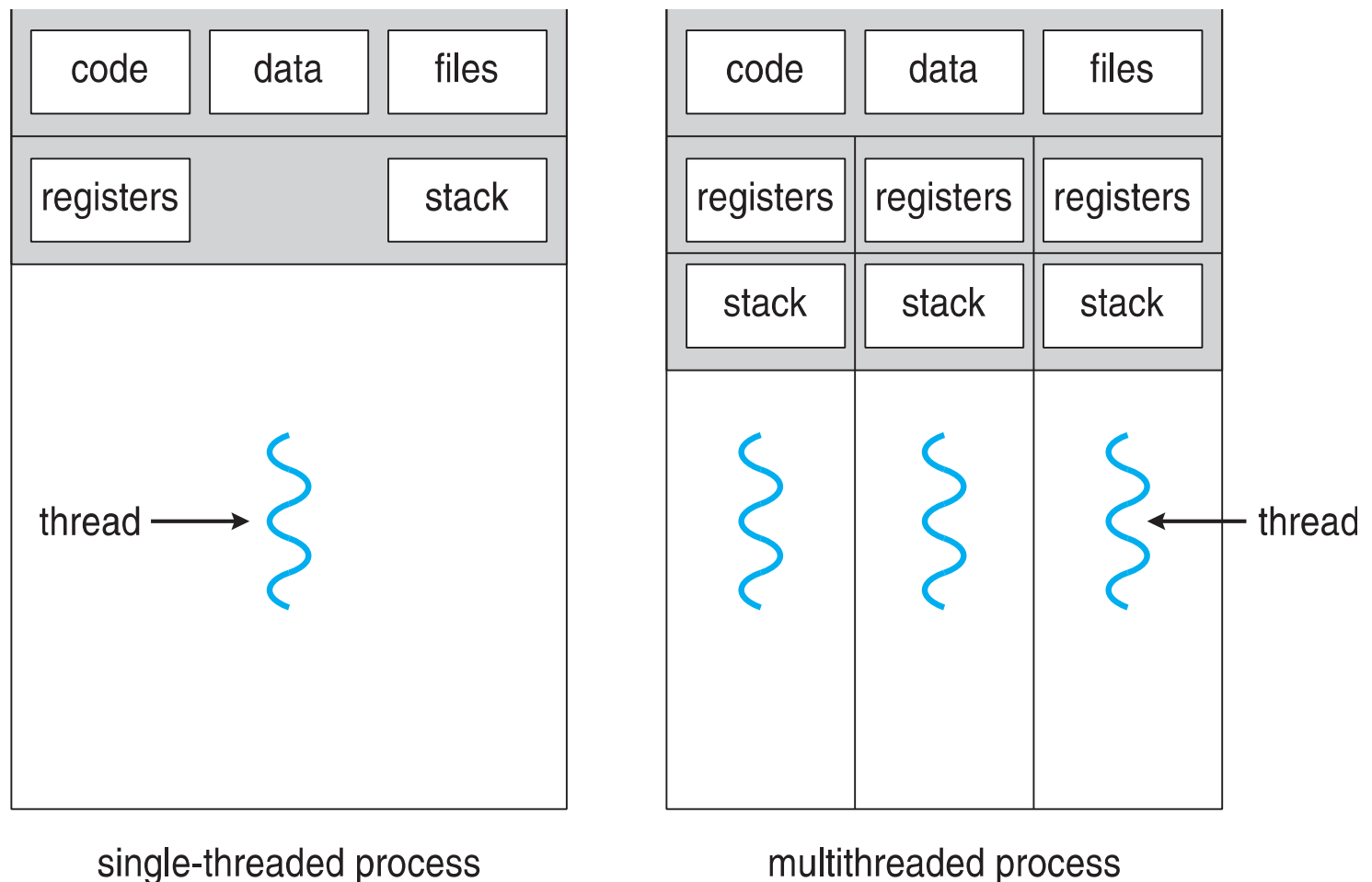
- Il context switch tra processi richiede molto lavoro al SO: oltre a cambiare il valore dei vari registri, deve spostarsi dalle aree dati e di codice del processo uscente, a quelle del processo entrante.
- Se dati e codice di quest'ultimo erano stati swappati in memoria secondaria, occorre prima riportarli in memoria primaria.
- Se due (o più) processi potessero condividere dati e codice, il context switch fra di loro sarebbe molto più veloce
- Per soddisfare questo tipo di esigenza è nato il concetto di thread

Motivazioni

- Le applicazioni sono generalmente multithreaded
- Task multipli in applicazioni implementati da thread separati
 - Update del display
 - Fetch data
 - Spell checking
 - Risposta in rete
- La creazione di processo è pesante, il thread è leggero
- Il kernel è generalmente multithreaded

Processi Single e Multithreaded

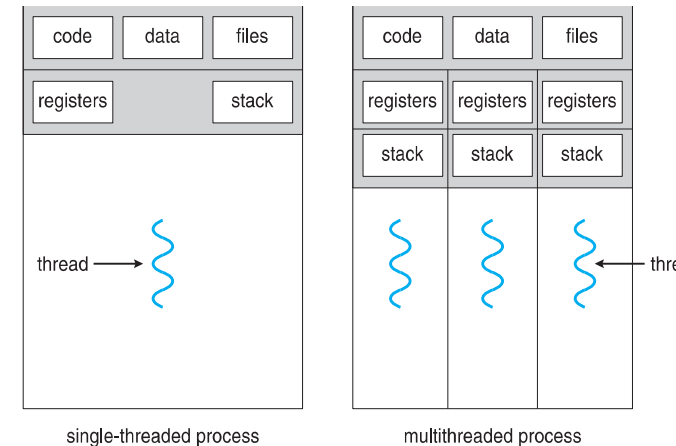
- Unità base di utilizzo di CPU
 - Caratterizzati da Thread ID, PC, Registri e Stack
 - Minima identità di task



Processi Single e Multithreaded

Unità base di utilizzo di CPU

- Caratterizzati da Thread ID, PC, Registri e Stack
- Minima identità di task



Ad ogni thread è associato in modo esclusivo il suo stato della computazione, fatto da:

- valore del program counter e degli altri registri della CPU
- uno stack

Ma un thread condivide con i suoi peer thread:

- il codice in esecuzione
- i dati
- i file aperti

Processi Single e Multithreaded

I **thread** sono “**unità esecutive**” indipendenti all'interno di un processo, caratterizzate da:

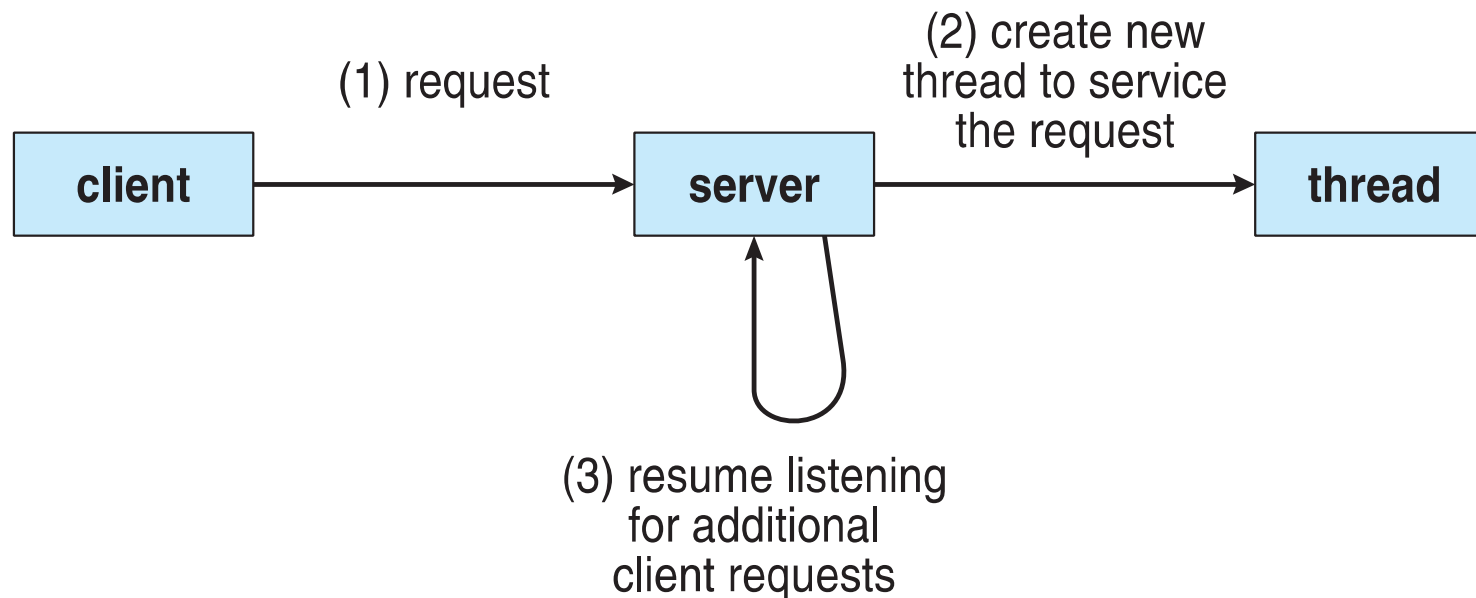
- un **thread ID**;
- un **insieme di registri**, incluso un contatore di programma ed un puntatore di pila;
- una **pila** per le variabili locali e gli indirizzi di ritorno (di chiamate a funzioni);
- una **maschera dei segnali**;
- una **priorità**.

Tutti i thread relativi ad uno stesso processo ne condividono:

- lo **spazio di indirizzamento**;
- i **descrittori dei file** aperti;
- la **disposizione** ed i **gestori dei segnali**;
- le **credenziali**.

Architettura di Server Multithread

- Molto utilizzate nelle architetture client-server
 - Per ogni client si genera un thread che gestisce il client
 - Per ogni richiesta di un client un thread
 - Server può gestire tante richieste eseguendo molti task leggeri



Benefici

■ Risposta

- Esecuzione continua se una parte del processo è bloccata (importante per le interfacce utente)

■ Risorse

- Thread condividono le risorse di un processo, più semplice di shared memory o message passing

■ Economia

- Più leggero di una creazione di processo, il **thread switching** ha minore overhead del **context switching**

■ Scalabilità

- I processi possono avvantaggiarsi delle architetture multiprocessore

Benefici

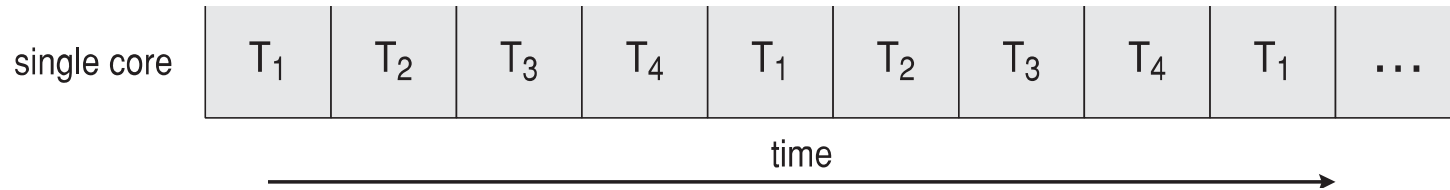
- I context switch avviene anche tra ognuno dei peer thread che formano task, in modo che tutti possano portare avanti la computazione
 - Ma il context switch fra peer thread richiede il salvataggio e il ripristino solo dei registri della CPU e dello stack (che sono diversi per ogni thread)
 - Codice, dati e file aperti sono gli stessi per tutti, e non devono essere cambiati: IL CONTEXT SWITCH TRA PEER THREAD E' MOLTO PIU' VELOCE

Multicore Programming

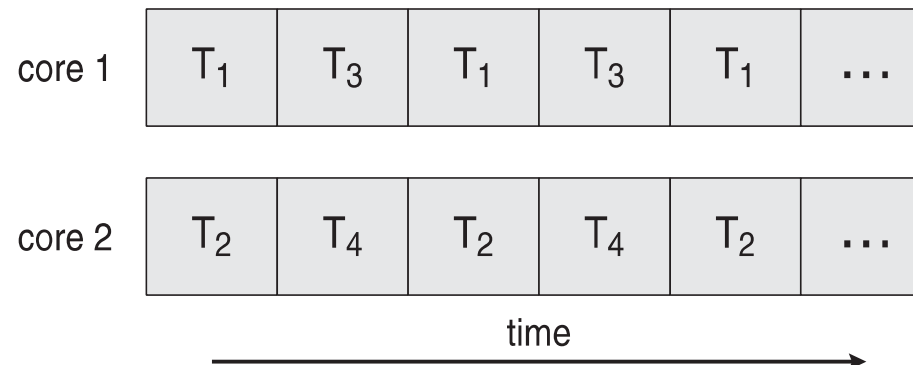
- Sistemi multitread forniscono meccanismi utili per sistemi multicore
- Sistemi **Multicore** o **multiprocessori** pongono diversi problemi:
 - **Come dividere le attività sui core**
 - Computazioni parallelizzabili su più core
 - **Come bilanciare i calcoli sui core**
 - **Come splittare i dati**
 - **Come gestire le dipendenze dei dati sui core**
 - Dipendenze possono richiedere sincronizzazioni
 - **Come fare test e debugging**
 - Più tracce di esecuzioni da testare
- **Parallelismo**
 - Più di un task simultaneamente distribuiti su più unità di calcolo
- **Concorrenza**
 - Consente il progresso contemporaneo di più di un task
 - Se processore/core singolo, concorrenza data dallo scheduler che intrallaccia le esecuzioni

Concorrenza vs. Parallelismo

□ Esecuzione concorrente su sistema single-core:

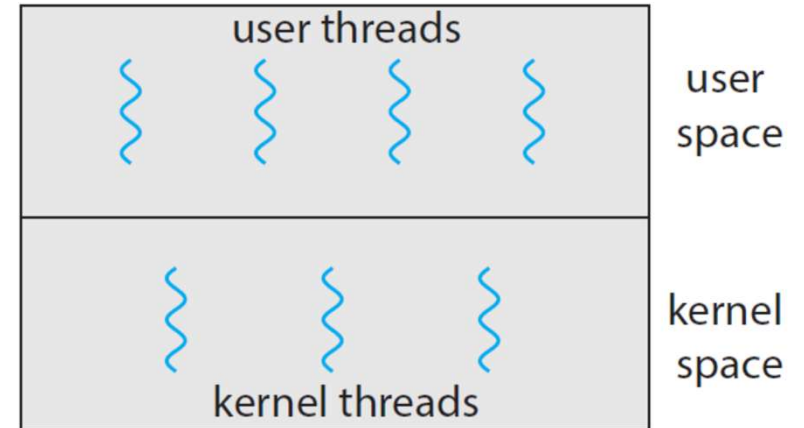


□ Parallelismo su sistema multi-core:



User Thread e Kernel Thread

- **User thread** – gestita da librerie di thread di livello utente
- In UNIX:
 - POSIX **Pthreads**



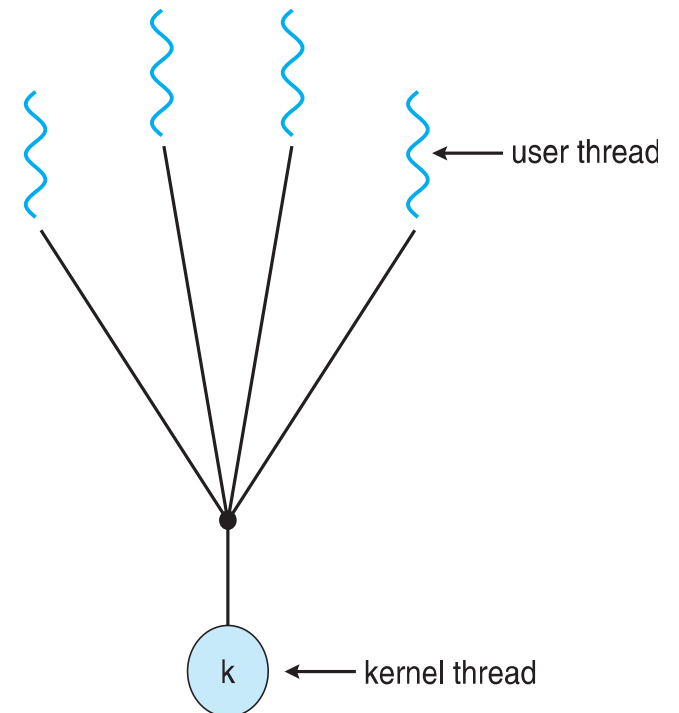
- **Kernel thread** - supportati dal Kernel
- Esempi – tutti gli SO general purpose inclusi:
 - Windows
 - Solaris
 - Linux
 - Tru64 UNIX
 - Mac OS

Modelli di Multithreading

- Diversi modi di gestire la relazione tra User e Kernel thread
 - Many-to-One
 - One-to-One
 - Many-to-Many

Many-to-One

- Tanti thread user mappati su un singolo kernel thread
- Però un thread bloccante (es. durante una system call) può bloccare tutto
- Problemi di parallelismo con sistemi multicore perché uno solo alla volta è in esecuzione nel Kernel
- Pochi sistemi utilizzano questo approccio:
 - **Solaris Green Threads** library
 - **GNU Portable Threads** library



Many-to-One

- Tanti thread user mappati su un singolo kernel thread
 - Lato kernel un solo PCB per gestire i thread di un processo

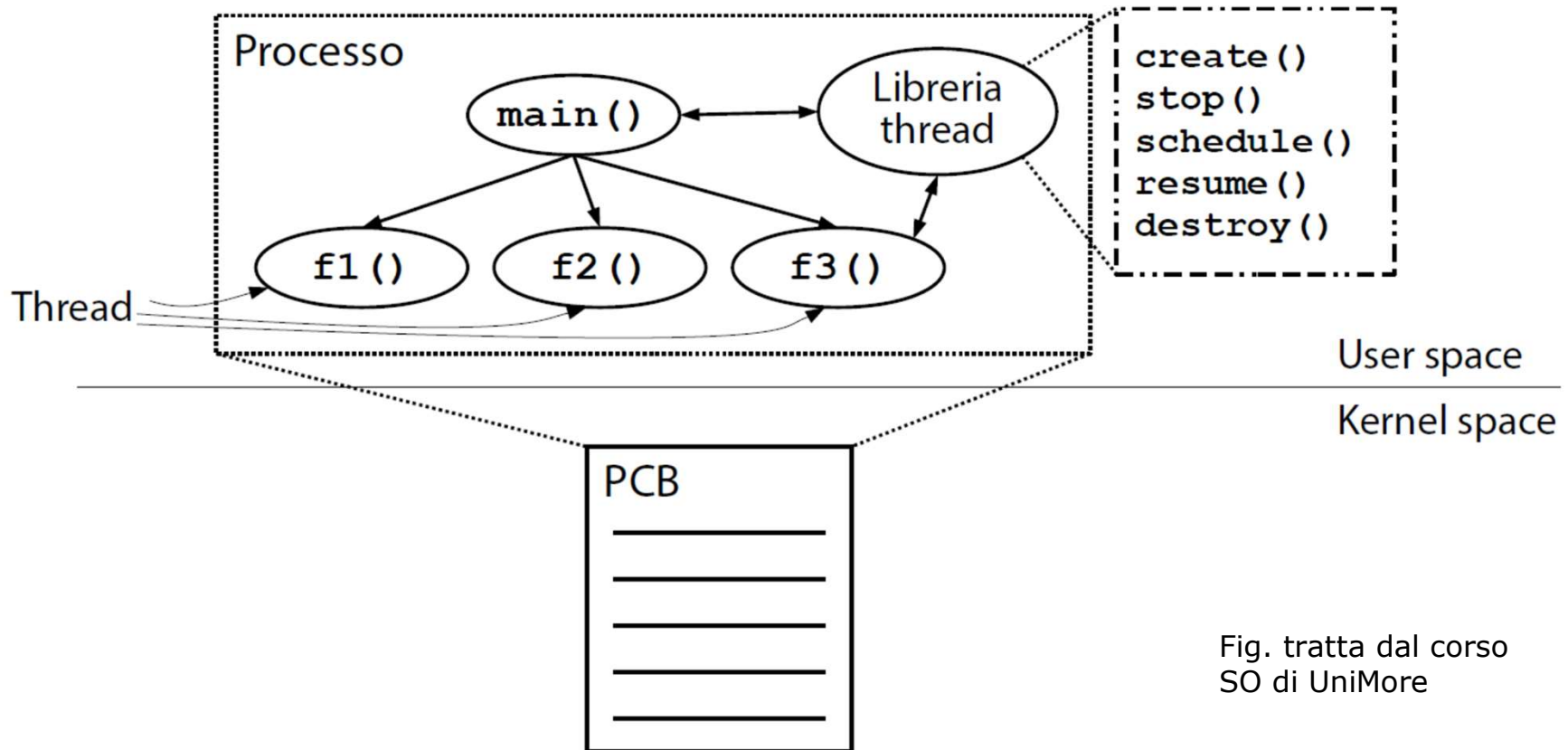
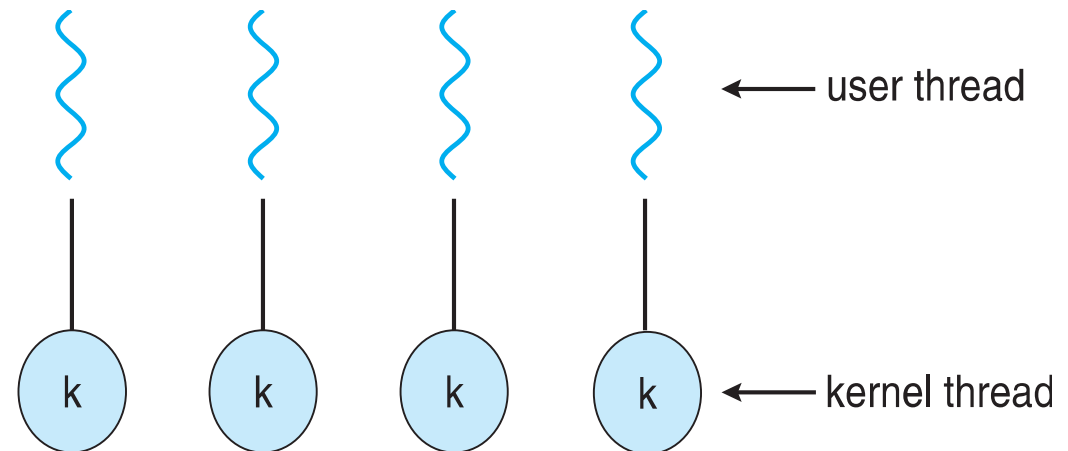


Fig. tratta dal corso
SO di UniMore

One-to-One

- Ogni thread user-level mappato su kernel thread
 - La creazione di thread user-level crea un kernel thread
 - Maggiore concorrenza di many-to-one
 - Il numero di thread per processo può essere limitato per evitare overhead
 - Esempi:
 - Windows
 - Linux
 - Solaris 9 e successivi
 - Utente deve controllare il numero dei thread



One-to-One

- Ogni thread user-level mappato su kernel thread
 - La creazione di thread user-level crea un kernel thread

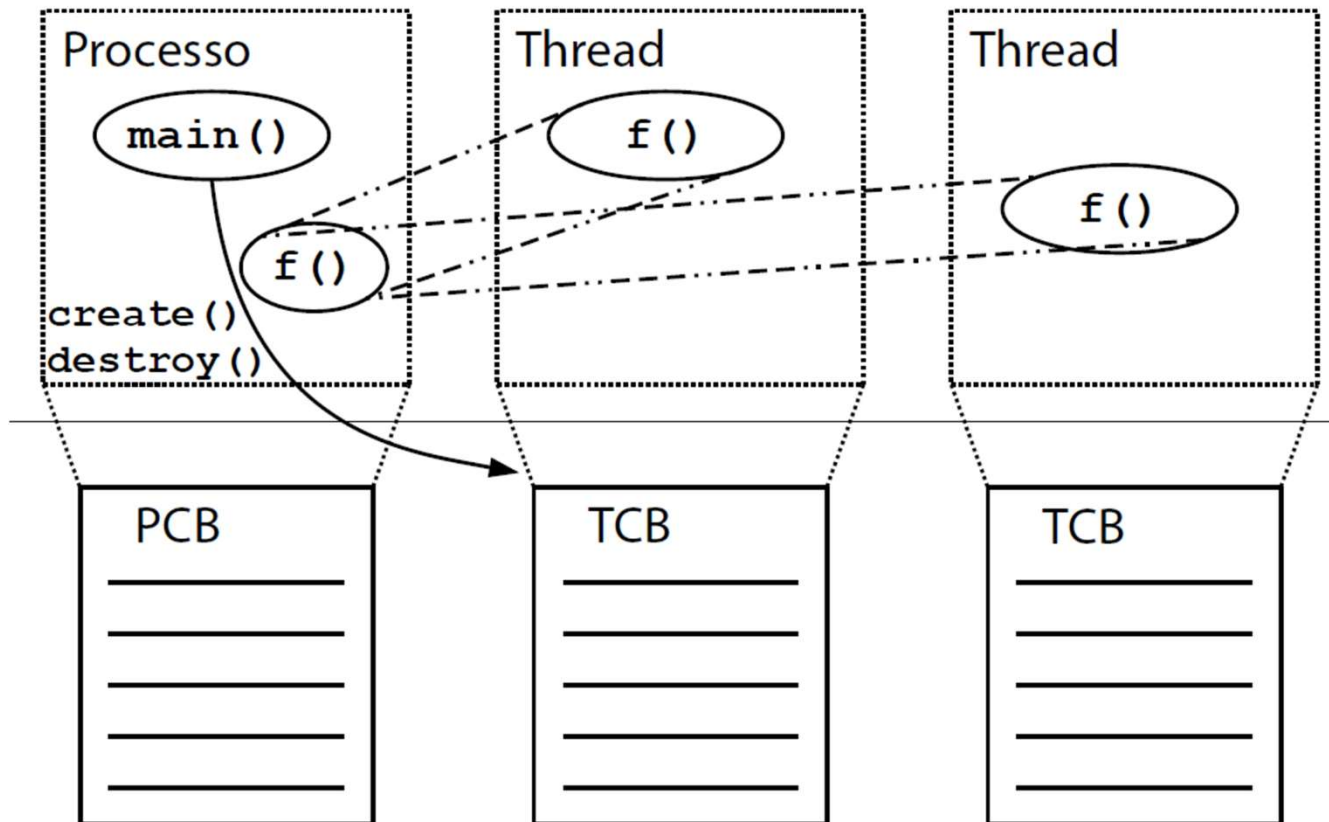
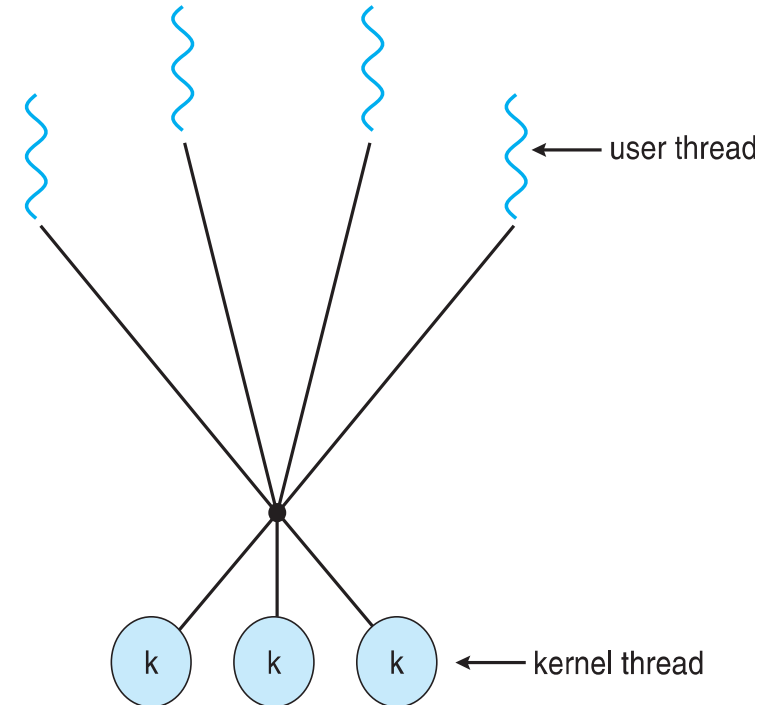


Fig. tratta dal corso
SO di UniMore

Many-to-Many

- Molti thread user-level sono mappati su molti thread del kernel
- Il SO può creare un numero sufficiente di thread del kernel
 - Però può gestirne il numero e il grado di parallelismo

- Il numero può dipendere dalla macchina e dall'applicazione
- L'utente può lanciare più thread, poi controlla il Sistema quanti thread kernel



Many-to-Many

- Molti thread user-level sono mappati su molti thread del kernel
- Il SO può creare un numero sufficiente di thread del kernel
 - Però può gestirne il numero e il grado di parallelismo

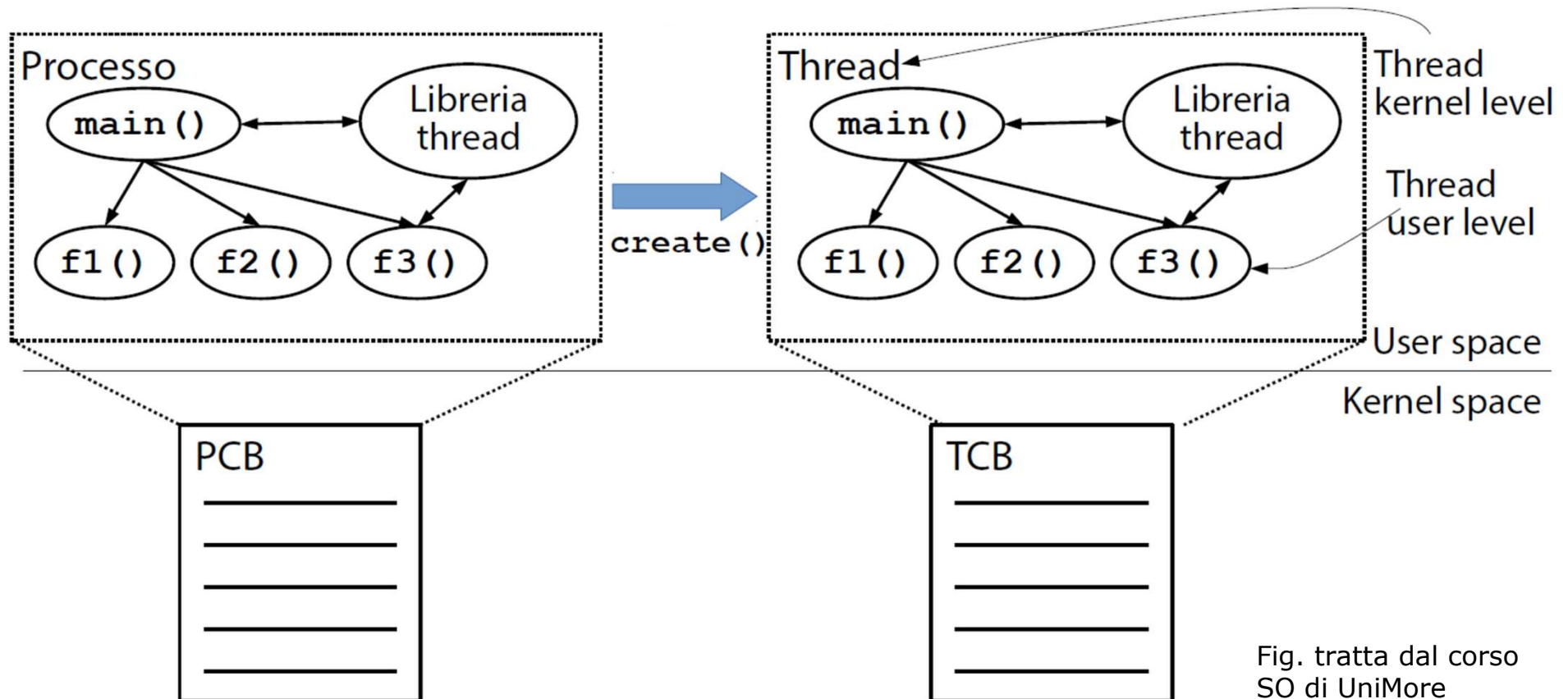
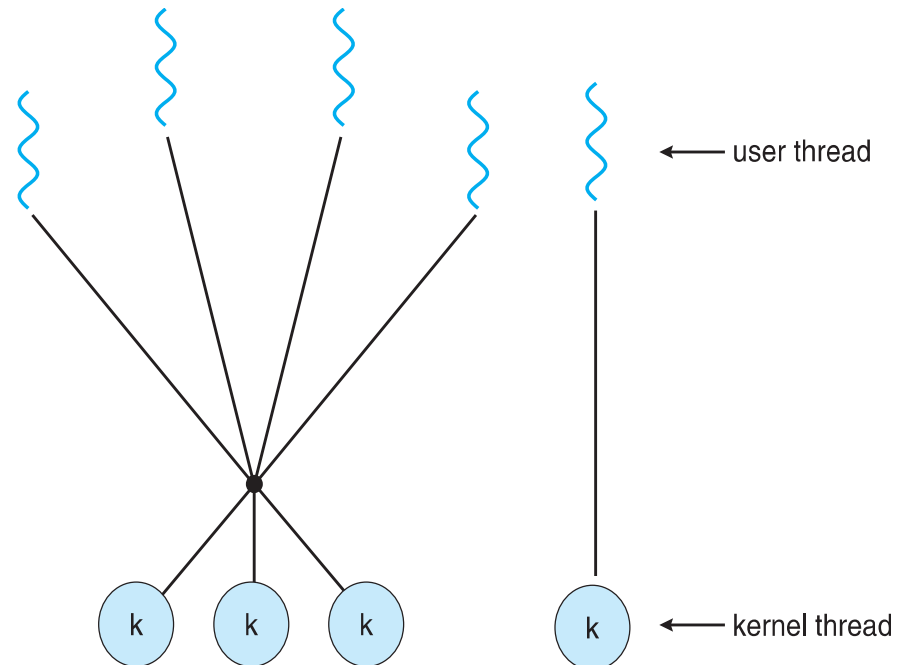


Fig. tratta dal corso
SO di UniMore

Two-level Model

- Variante del Many-to-Many che permette ad un user-thread di essere legato (**bound**) ad un kernel thread
- Esempi:
 - IRIX
 - HP-UX
 - Tru64 UNIX
 - Solaris 8 e precedenti



Librerie per Thread

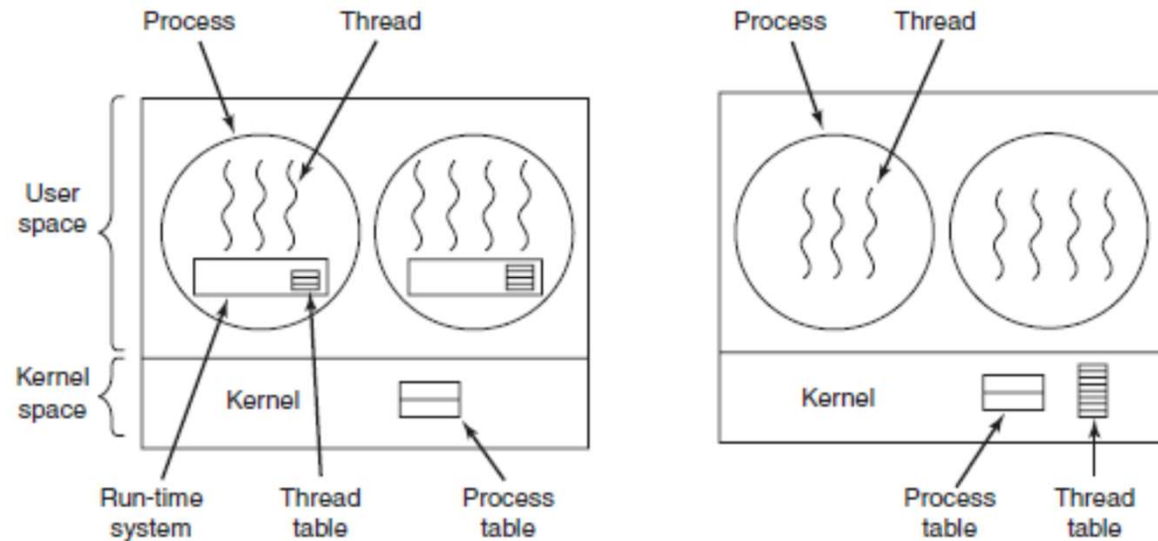
- **Librerie per i thread (Thread library)**
 - API per creare e gestire i thread

- Due modi principali di implementarle
 - Libreria interamente nello **user space**
 - I thread sono implementati al livello utente (gestiti con chiamate di funzione utente)

 - Libreria al **livello Kernel** supportata dal SO
 - I thread sono gestiti con chiamate di Sistema (gestiti con chiamate di sistema)

Librerie per Thread

- **Librerie per i thread (Thread library)**
 - API per creare e gestire i thread
- Due modi principali di implementarle
 - Libreria interamente nello **user space**
 - Libreria al **livello Kernel** supportata dall'SO



Pthreads

- Estensione di POSIX standard (IEEE 1003.1c) fornisce le API per la creazione e sincronizzazione dei thread
- Esiste sia in versione user-level sia kernel-level
- **Specifica, non implementazione**
- API specifica il comportamento delle funzioni di libreria, implementazione lasciata allo sviluppatore
- Tipiche in UNIX (Solaris, Linux, Mac OS)

Identificare i Thread

- processo ha `process id (pid)` di tipo `pid_t`
- thread ha `thread id (tid)` di tipo `pthread_t`

Nota: `pthread_t` struttura, funzione per il confronto:

```
int pthread_equal(pthread_t t1, pthread_t t2);
```

Ritorna non zero se uguali, 0 se diversi

```
pthread_t pthread_self(void);
```

Restituisce il tid del thread corrente

Creazione Thread

- Analogamente ai processi, quando un thread viene creato esso è associato ad un pezzo di codice;
- Tuttavia solo se il processo ospite (e quindi il suo corrispondente programma) è single-threaded, il thread è associato all'intero programma;
- Se il processo ospite è multi-threading, ciascun thread di tale processo è associato ad una funzione da eseguire, detta **funzione di avvio**;
- Anche i thread, come i processi, possono assumere gli stati **running**, **sleeping**, **blocked** e **terminated** ;
- Quando un thread termina la situazione è analoga al caso dei processi: è il programmatore che deve preoccuparsi affinché tutte le risorse impegnate dal thread siano rilasciate al sistema, facendo in modo che venga effettuata una **wait** a livello di thread.

Creazione Thread

- Quando un programma viene mandato in esecuzione tramite una chiamata **exec**, viene creato un singolo thread, detto **thread principale** o **iniziale**;
- Ulteriori thread vanno creati esplicitamente;
- Ogni thread ha numerosi **attributi**, da assegnare alla creazione: **livello di priorità**, **dimensione iniziale della pila**, etc;
- All'atto della creazione di un thread è necessario specificare la **funzione di avvio**: il thread inizierà la propria esecuzione richiamando la funzione di avvio;
- La **terminazione** di un thread può avvenire in tre diverse circostanze: **(a)** una chiamata esplicita di terminazione del thread, **(b)** la funzione di avvio ritorna, **(c)** il processo contenente il thread termina.

Pthreads

Per creare thread aggiuntionali relativi ad uno stesso processo, Posix prevede la funzione:

```
#include <pthread.h>

int pthread_create ( pthread_t *tid, const pthread_attr_t *attr,
                    void * ( *start_func) (void *), void *arg );
```

- se la chiamata ha successo, *tid* punta al thread ID;
- *attr* permette di specificare gli attributi del thread (se *attr* = NULL, gli attributi sono quelli di default);
- *start_func* è l'indirizzo della funzione di avvio;
- *arg* è l'indirizzo dell'argomento accettato dalla funzione di avvio;
- restituisce 0 in caso di successo, un intero positivo – secondo le convenzioni di `<sys/errno.h>` – in caso di errore.

Pthreads

```
typedef void (*thread_start)(void *);
```

```
int pthread_create(pthread_t      *tid,  
                  const pthread_attr_t *attributes  
                  thread_start    start,  
                  void             *argument);
```

- Restituisce 0 se OK, un codice d'errore altrimenti
- tid = argomento di ritorno, conterrà il tid del nuovo thread
- attributes = attributi del thread (vedere dopo)
- start = indirizzo della funzione da cui partire
- argument = l'argomento passato alla funzione start

Pthreads

- invocare `exit()` (`_exit`, `_Exit`) fa terminare *l'intero processo!*
- Analogamente un segnale ad un thread uccide il processo
- per terminare solo il thread corrente, si puo':
 - invocare `return` dalla routine di `start`, il valore di ritorno e' l'exit code
 - invocare `pthread_exit`
 - un altro thread del processo puo' chiamare `pthread_cancel`

Pthreads

Un thread può richiedere esplicitamente la propria terminazione grazie alla chiamata seguente, lasciando traccia del proprio stato di terminazione per quei thread che attendono per lui:

```
#include <pthread.h>

void pthread_exit ( void *status);
```

- *status* punta all'oggetto che definisce lo stato di terminazione del thread. Quest'ultimo **non** deve essere una variabile **locale** al thread chiamante, pena la sua scomparsa alla terminazione del thread stesso.

Pthreads

```
void pthread_exit(void *status);
```

- termina il thread corrente, con valore di uscita status
- altri thread possono raccogliere il valore di uscita usando `pthread_join` (vedere slide successiva)
- fare attenzione che i dati puntati da ret sopravvivano alla terminazione del thread!
 - status non deve puntare allo stack (no variabili locali)
 - Ok uso di variabili globali o allocate dinamicamente

Pthreads

Un thread può attendere per la terminazione di un altro thread relativo allo stesso processo:

```
#include <pthread.h>
```

```
int pthread_join ( pthread_t *tid, void **status );
```

- *tid* è l'ID del thread del quale si vuole attendere la terminazione;
- *status* punta al valore restituito dal thread per cui si è atteso, indicante il suo stato di terminazione (se *status* = NULL, tale stato non viene restituito);
- restituisce 0 in caso di successo, un intero positivo – secondo le convenzioni di `<sys/errno.h>` – in caso di errore.

Pthreads

```
int pthread_join(pthread_t tid, void **ret);
```

- attende che il thread specificato da tid termini
 - se quel thread e' gia' terminato, ritorna subito (come wait)
- restituisce 0 se OK, un codice d'errore altrimenti
- ret e' un parametro di ritorno usato per restituire il valore d'uscita della funzione di start del thread atteso (return), se il thread e' cancellato contiene PTHREAD_CANCELED
- se il valore di uscita non ci interessa, passiamo NULL al posto di ret

Esempio Pthreads

- Esempio somma dei primi N numeri
 - Per sempio per $N = 5$, $\text{Sum} = 1 + 2 + 3 + 4 + 5 = 15$
 - Si può delegare il calcolo ad uno o più thread
 - Abbiamo thread **sincroni** e **asincroni**
 - ▶ Nel caso sincrono il genitore aspetta i figli
 - ▶ Nel caso asincrono il genitore continua subito

Esempio Pthreads

```
#include <pthread.h>
#include <stdio.h>

int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* threads call this function */

int main(int argc, char *argv[])
{
    pthread_t tid; /* the thread identifier */
    pthread_attr_t attr; /* set of thread attributes */

    if (argc != 2) {
        fprintf(stderr, "usage: a.out <integer value>\n");
        return -1;
    }
    if (atoi(argv[1]) < 0) {
        fprintf(stderr, "%d must be >= 0\n", atoi(argv[1]));
        return -1;
    }
}
```


Esempio Pthreads

```
    /* get the default attributes */
    pthread_attr_init(&attr);
    /* create the thread */
    pthread_create(&tid,&attr,runner,argv[1]);
    /* wait for the thread to exit */
    pthread_join(tid,NULL);

    printf("sum = %d\n",sum);
}

/* The thread will begin control in this function */
void *runner(void *param)
{
    int i, upper = atoi(param);
    sum = 0;

    for (i = 1; i <= upper; i++)
        sum += i;

    pthread_exit(0);
}
```

Pthreads

```
/* thread_create: stampa i TID del main thread e di due altri
thread */

#include <pthread.h>
#include <stdio.h>
#include <errno.h>

void *start_func(void *arg) /* funzione di avvio */
{
    printf("%s", (char *)arg);
    printf(" and my TID is: %d\n", (int)pthread_self());
}

int main(void)
{
    int en;
    pthread_t tid1, tid2;
    char *msg1 = "Hello world, I am thread #1";
    char *msg2 = "Hello world, I am thread #2";

    printf("The launching process has PID:%d\n", (int)getpid());

    printf("The main thread has TID:%d\n", (int)pthread_self());
```


Pthreads

```
/* thread_create: stampa i TID del main thread e di due altri
thread */

#include <pthread.h>
#include <stdio.h>
#include <errno.h>

void *start_func(void *arg) /* funzione di avvio */
{
    printf("%s", (char *)arg);
    printf(" and my TID is: %d\n", (int)pthread_self());
}

int main(void)
{
    int en;
    pthread_t tid1, tid2;
    char *msg1 = "Hello world, I am thread #1";
    char *msg2 = "Hello world, I am thread #2";

    printf("The launching process has PID:%d\n", (int)getpid());

    printf("The main thread has TID:%d\n", (int)pthread_self());
```

Pthreads

```
/* crea il 1mo thread */
if ((en = pthread_create(&tid1, NULL, start_func, msg1) != 0))
    errno=en, perror("pthread_create"), exit(1);

/* crea il 2ndo thread */
if ((en = pthread_create(&tid2, NULL, start_func, msg2) != 0))
    errno=en, perror("pthread_create"), exit(2);

/* attende per il 1mo */
if ((en = pthread_join(tid1, NULL) != 0))
    errno=en, perror("pthread_join"), exit(1);

/* attende per il 2ndo */
if ((en = pthread_join(tid2, NULL) != 0))
    errno=en, perror("pthread_join"), exit(2);

return 0;
}
```

Pthreads

```
#include <pthread.h>
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

int count = 0;

void *f(void *x)
{
    sleep(rand() % 10);
    count++;
    printf("Ciao! %d\n", count);
    return NULL;
}
```

Pthreads

```
#include <pthread.h>
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

int count = 0;

void *f(void *x)
{
    sleep(rand() % 10);
    count++;
    printf("Ciao! %d\n", count);
    return NULL;
}
```

```
int main(int argc, char *argv[])
{
    int i, err, n;

    if (argc != 2) {
        printf("Uso: %s <numero thread>\n", argv[0]);
        exit(1);
    }

    n = atoi(argv[1]);
    /* Allocazione consentita */
    pthread_t tid[n];

    for (i=0; i<n ;i++) {
        if ((err=pthread_create(&tid[i], NULL, f, NULL)) != 0) {
            printf("errore: %s\n", strerror(err));
            exit(1);
        }
    }

    for (i=0; i<n ;i++)
        pthread_join(tid[i],NULL);
    printf("finito.\n");

    return 0;
}
```

Passaggio di Parametri

```
#include <pthread.h>
#include <stdio.h>

void *tbody(void *arg)
{
    int j;
    printf(" ciao sono un thread, mi hanno appena creato\n");
    *(int *)arg = 10;
    sleep(2)      /* faccio aspettare un pò il mio creatore, poi termino */
    pthread_exit((int *)50); /* oppure return ((int *)50); */
}
```

funzione che contiene il
codice di un peer thread

Funzione di avvio

```
main(int argc, char **argv)
{
    int i;
    pthread_t mythread;
    void *result;
    printf("sono il primo thread, ora ne creo un altro \n");
    pthread_create(&mythread, NULL, tbody, (void *) &i);
    printf("ora aspetto la terminazione del thread che ho creato \n");
    pthread_join(mythread, &result);
    printf("Il thread creato ha assegnato %d ad i\n", i);
    printf("Il thread ha restituito %d \n", result);
}
```

Passa &i al thread, Nota: void * nella start permette di passare strutture di diverso tipo

Legge &result dal thread, Nota: void * restituito dalla start permette di leggere strutture di diverso tipo

Similitudini thread-processi

- fork
- exit
- waitpid
- kill
- getpid
- processo zombie
- pthread_create
- pthread_exit
- pthread_join
- pthread_kill
- pthread_self
- thread terminato in attesa di pthread_join

Condivisione di Memoria

- I due thread condividono lo stesso spazio di indirizzamento, e quindi vedono le stesse variabili: se uno dei due modifica una variabile, la modifica è vista anche dall'altro thread.
- Nel codice precedente *il main passa al thread tbody il puntatore alla variabile i dichiarata nel main*. il thread tbody modifica la variabile, e questa modifica è vista da main.
- Nel caso dei processi tradizionali, una cosa simile è ottenibile solo usando esplicitamente un segmento di memoria condivisa.

Condivisione di Memoria

- Ma i thread di un task possono condividere variabili in maniera ancora più semplice, usando variabili globali.

```
#include <pthread.h>
#include <stdio.h>

int global_var = 5;

void *tbody(void *arg)
{
    printf(" ciao sono un thread, ora modifico una var globale\n");
    global_var = 27;
    *(int *)arg = 10;
    pthread_exit((int *)50); /* oppure return ((int *)50); */
}
```


Pthreads

```
typedef struct foo{  
    int a;  
    int b;  
} myfoo;
```

```
myfoo test; // Variabile GLOBALE
```

```
void stampa(char *st, struct foo *test){  
    printf("%s: tid=%d a=%d b=%d\n", st, pthread_self(), test->a, test->b);  
}
```

```
void *fun1(void *arg){  
    myfoo test2 = {1,2}; // Variabile LOCALE  
    printf("%s %d\n", arg, pthread_self());  
    stampa(arg, &test2);  
    pthread_exit((void *)&test2);  
}
```

Pthreads

```
void *fun2(void *arg){
    test.a = 3;
    test.b = 4; // Variabile GLOBALE
    printf("%s %d\n", arg, pthread_self());
    stampa(arg, &test);
    pthread_exit((void *)&test);
}
```

```
void *fun3(void *arg){
    myfoo *test3;
    test3=malloc(sizeof(struct foo)); // Variabile allocata dinamicamente
    test3->a = 5;
    test3->b = 6;
    printf("%s %d\n", arg, pthread_self());
    stampa(arg, test3);
    pthread_exit((void *)test3); //c
}
```

Pthreads

```
int main(void){
    char st[100];
    pthread_t tid1;
    pthread_t tid2;
    pthread_t tid3;

    myfoo *b; // PUNTATORE alla struttura (non allocata)

    pthread_create(&tid1, NULL, fun1, "Thread 1"); // Locale
    pthread_join(tid1, (void *)&b);
    stampa("Master ", b);

    pthread_create(&tid2, NULL, fun2, "Thread 2"); // Globale
    pthread_join(tid2, (void *)&b);
    stampa("Master ", b);

    pthread_create(&tid3, NULL, fun3, "Thread 3"); // Dinamica
    pthread_join(tid3, (void *)&b);
    stampa("Master ", b);
}
```

Pthreads

Thread 1: 1077283760

// Locale

Thread 1: a=1 b=2

Master : a=1075156600 b=1077281896

Thread 2: 1077283760

// Globale

Thread 2: a=3 b=4

Master : a=3 b=4

Thread 3: 1077283760

// Dinamica

Thread 3: a=5 b=6

Master : a=5 b=6

Problematiche Threading

- Semantica delle chiamate **fork()** ed **exec()**
- Signal handling
 - Sincrono e asincrono
- Cancellazione thread
 - Asincrono o deferred
- Thread-local storage
- Attivazione Scheduler

Semantica di `fork()` ed `exec()`

- **`fork()`** duplica solo il thread chiamante o tutti i thread del processo?
 - Per alcune versioni di UNIX due versioni di `fork`
 - Spesso duplica solo il thread chiamante
- **`exec()`** funziona normalmente – sovrascrive il processo in esecuzione inclusi i thread

Segnali

- Le chiamate a signal influenzano tutti i thread
- Se arriva un segnale a un processo, succede che:
 - se il processo ha impostato un handler, il segnale arriva ad *uno qualunque* dei thread (che esegue l'handler)
 - se invece la reazione al segnale consiste nel terminare il processo, *tutti i thread* vengono terminati

Segnali

```
int pthread_kill(pthread_t tid, int signo);
```

- manda il segnale signo al thread specificato da tid
 - se e' impostato un handler, viene eseguito nel thread tid
 - se non e' impostato un handler, e il comportamento di default e' di terminare il processo, vengono comunque terminati tutti i thread
- restituisce 0 se OK, un codice d'errore altrimenti